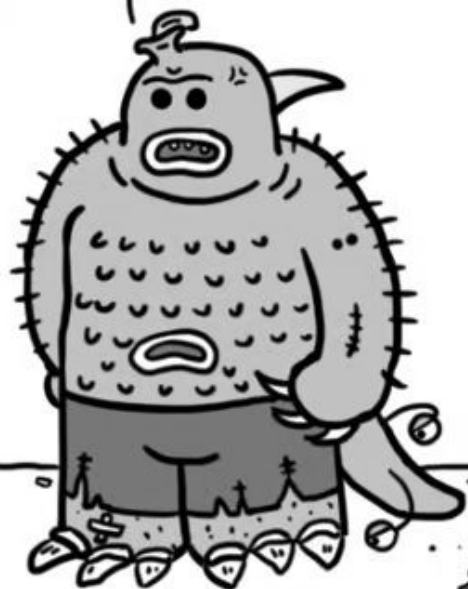

Refactoring

INFO248

Dra. Valeria Henríquez

So, you're
my code?
Why are you
here this way?

You really
have no clue?



REFACTOR ME!!



¿Qué es Refactoring?

El objetivo principal de la refactorización es luchar contra la deuda técnica. Transforma un desastre en código limpio y con un diseño simple.

La refactorización es una técnica disciplinada para reestructurar un cuerpo de código existente, alterando su estructura interna sin cambiar su comportamiento externo.

Sustantivo: un cambio realizado en la estructura interna del software para que sea más fácil de entender y más barato para modificar sin cambiar su comportamiento observable.

Verbo: reestructurar el software aplicando una serie de refactorizaciones sin cambiar su comportamiento.

The Addison-Wesley Signature Series

"Any fool can write code that a computer can understand.
Good programmers write code that humans can understand."

—M. Fowler (1999)



REFACTORING

Improving the Design of Existing Code

Martin Fowler

with contributions by
Kent Beck



SECOND EDITION



Robert C. Martin Series



Clean Code

A Handbook of Agile Software Craftsmanship



Robert C. Martin



Foreword by James O. Coplien

Tags

- ☐ basic
- ☐ encapsulation
- ☐ moving-features
- ☐ organizing-data
- ☐ simplify-conditional-logic
- ☐ refactoring-apis
- ☐ dealing-with-inheritance
- ☐ collections
- ☐ delegation
- ☐ errors
- ☐ extract
- ☐ parameters
- ☐ fragments
- ☐ grouping-function
- ☐ immutability
- ☐ inline
- ☐ remove
- ☐ rename
- ☐ split-phase
- ☐ variables

#

Change Function Declaration

Add Parameter • Change Signature • Remove Parameter • Rename Function • Rename Method

Change Reference to Value

Change Value to Reference

Collapse Hierarchy

Combine Functions into Class

Combine Functions into Transform

Consolidate Conditional Expression

Decompose Conditional

Encapsulate Collection

Encapsulate Record

Replace Record with Data Class

Encapsulate Variable

Encapsulate Field • Self-Encapsulate Field

Remove Dead Code

Remove Flag Argument

Replace Parameter with Explicit Methods

Remove Middle Man

Remove Setting Method

Remove Subclass

Replace Subclass with Fields

Rename Field

Rename Variable

Replace Command with Function

Replace Conditional with Polymorphism

Replace Constructor with Factory Function

Replace Constructor with Factory Method

Replace Control Flag with Break

Rename Variable

name
nm



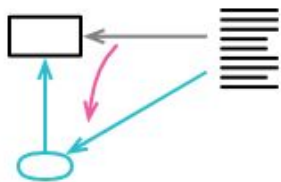
```
let a = height * width;
```



```
let area = height * width;
```

¿Qué pros y cons
identifican?

Encapsulate Variable



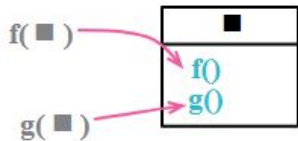
```
let defaultOwner = {firstName: "Martin", lastName: "Fowler"};
```



```
let defaultOwnerData = {firstName: "Martin", lastName: "Fowler"};  
export function defaultOwner() {return defaultOwnerData;}  
export function setDefaultOwner(arg) {defaultOwnerData = arg;}
```

¿Qué pros y cons
identifican?

Combine Functions into Class



```
function base(aReading) {...}
function taxableCharge(aReading) {...}
function calculateBaseCharge(aReading) {...}
```



```
class Reading {
  base() {...}
  taxableCharge() {...}
  calculateBaseCharge() {...}
}
```

Cuando existe un grupo de funciones que operan estrechamente juntas en un cuerpo común de datos (normalmente pasados como argumentos para la llamada a la función), hay una oportunidad de formar una clase.

El uso de una clase hace que el entorno común que comparten estas funciones sea más explícito, permite simplificar las llamadas a funciones dentro del objeto eliminando muchos de los argumentos y proporciona una referencia para pasar dicho objeto en otras partes del sistema.

¿Qué pros y cons identifican?

Introduce Parameter Object



```
function amountInvoiced(startDate, endDate) {...}  
function amountReceived(startDate, endDate) {...}  
function amountOverdue(startDate, endDate) {...}
```



```
function amountInvoiced(aDateRange) {...}  
function amountReceived(aDateRange) {...}  
function amountOverdue(aDateRange) {...}
```

¿Qué pros y cons
identifican?

Inline Variable



```
let basePrice = anOrder.basePrice;  
return (basePrice > 1000);
```



```
return anOrder.basePrice > 1000;
```

¿Qué pros y cons
identifican?

Extract Variable



```
return order.quantity * order.itemPrice -  
    Math.max(0, order.quantity - 500) * order.itemPrice * 0.05 +  
    Math.min(order.quantity * order.itemPrice * 0.1, 100);
```



```
const basePrice = order.quantity * order.itemPrice;  
const quantityDiscount = Math.max(0, order.quantity - 500) * order.itemPrice * 0.05;  
const shipping = Math.min(basePrice * 0.1, 100);  
return basePrice - quantityDiscount + shipping;
```

¿Qué pros y cons
identifican?

¿Cuándo refactorizar?

Regla de tres ("*Tres strikes y tú refactorizas*")

1. Cuando estés haciendo algo por primera vez, simplemente hazlo.
2. Cuando hagas algo similar por segunda vez, avergüénzate de tener que repetirlo, pero hazlo de todos modos.
3. **Cuando hagas algo por tercera vez, empieza a refactorizar.**



¿Cuándo refactorizar?

Al agregar una funcionalidad

- La refactorización ayuda a comprender el código de otras personas. Si tienes que lidiar con código sucio de otra persona, intente refactorizarlo primero.
 - a. El código limpio es mucho más fácil de entender.
 - b. Lo mejorarás no solo para ti, sino también para aquellos que lo usen después de ti.
- La refactorización facilita la adición de nuevas funcionalidades. Es mucho más fácil hacer cambios en código limpio.

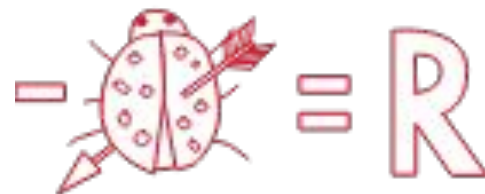


¿Cuándo refactorizar?

Al corregir un error

- Los errores en el código se comportan igual que en la vida real: viven en los lugares más oscuros y sucios del código. Limpia tu código y los errores prácticamente *se descubrirán solos*.
- Los jefes aprecian la refactorización proactiva, ya que elimina la necesidad de tareas especiales de refactorización más adelante.

¡Los jefes felices hacen felices a los programadores!



¿Cuándo refactorizar?

Durante una revisión de código

La revisión del código puede ser la última oportunidad para ordenar el código antes de que esté disponible para el público.

Lo mejor es realizar estas revisiones en pareja. De esta manera, podría solucionar problemas simples rápidamente y medir el tiempo para solucionar los más difíciles.



Código limpio

El código limpio es obvio para otros programadores: no hablamos de algoritmos súper sofisticados. Nos referimos a la mala nomenclatura de las variables, las clases y los métodos muy extensos, todo eso hace que el código sea descuidado y difícil de entender.

El código limpio no contiene duplicación: Cada vez que tenga que realizar un cambio en un código duplicado, debe recordar realizar el mismo cambio en cada instancia. Esto aumenta la carga cognitiva y ralentiza el progreso.

El código limpio contiene un número mínimo de clases y partes móviles: El código es una responsabilidad, manténgalo corto y simple.

El código limpio pasa todas las pruebas: Sabes que tu código está sucio cuando solo el 95% de tus pruebas resultan exitosas. Sabes que estás perdido cuando la cobertura de tus pruebas es del 0%.

¡El código limpio es más fácil y barato de mantener!

Pausa (5 min.)

Smell Codes

- What? How can code "smell"??
- Well it doesn't have a nose... but it definitely can stink!



Smell Codes - Bloaters

Los códigos inflados (*Bloaters*) son métodos y clases que han aumentado a proporciones tan gigantescas que es difícil trabajar con ellos. Por lo general, “estos olores” no aparecen de inmediato, sino que se acumulan con el tiempo a medida que evoluciona el software (y especialmente cuando nadie se esfuerza por erradicarlos).



```
public abstract class Animal {  
    private int edad;  
    private String sexo;  
    private String sonido;  
  
    public String sonido(Object animal) {  
        String sonido = "";  
  
        if (animal instanceof Mamifero) {  
            if (animal instanceof Gato) {  
                sonido = "Meow";  
            }  
        }  
  
        return sonido;  
    }  
}  
  
public abstract class Mamifero extends Animal { }  
  
public class Gato extends Mamifero { }
```

Smell Codes - Object-Orientation Abusers

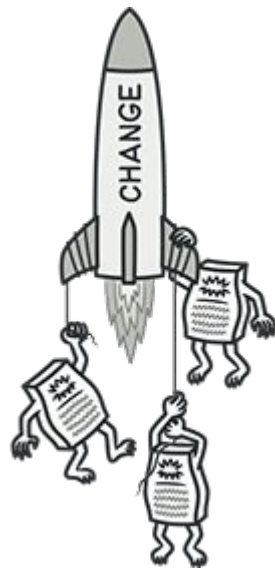
Los olores “Abusadores de la orientación a objetos” se refieren a la aplicación incompleta o incorrecta de los principios de la programación orientada a objetos.



```
public abstract class Animal {  
    private int edad;  
    private String sexo;  
    private String sonido;  
  
    private String clima;  
    private String temperaturaAire;  
    private String temperaturaAgua;  
    private String temperaturaSuelo;  
    private double humedad;  
}
```

Smell Codes - Change Preventers

Los olores “inhibidores de cambio” significan que si necesitas cambiar algo en un lugar de tu código, también tendrás que realizar muchos cambios en otros lugares. Como resultado, el desarrollo de programas se vuelve mucho más complicado y costoso.



```
public class Animal {
    private int edad;
    private String sexo;
    private String sonido;

    public void printarEdad() {
        System.out.println("Edad: " + this.edad);
    }
}

public class Mamifero {
    private int edad;
    private String sexo;
    private String sonido;

    public void printarEdadMamifero() {
        System.out.println("Edad: " + this.edad);
    }
}

public class Insecto {
    private int edad;
    private String sexo;
    private String sonido;

    public void printarEdadInsecto() {
        System.out.println("Edad: " + this.edad);
    }
}
```


Smell Codes - Dispensables

Los olores “dispensables/desechables” se refieren a la existencia de código prescindible; algo inútil e innecesario cuya ausencia haría que el código fuera más limpio, más eficiente y más fácil de entender.



Remove Dead Code



```
if(false) {  
    doSomethingThatUsedToMatter();  
}
```



```
void example1() {  
    int x = 10;  
    return; // Program exits here  
    x = 20; // Dead code (never executed)  
}
```

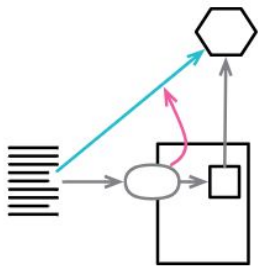
```
void example2() {  
    int x = 5; // Dead code (x is never used)  
    int y = 10;  
    printf("%d", y);  
}
```

Smell Codes - Couplers

Todos los olores “acopladores” contribuyen a un acoplamiento excesivo entre clases o muestran lo que sucede si el acoplamiento se reemplaza por una delegación excesiva.



Remove Middle Man



```
manager = aPerson.manager;
```

```
class Person {  
    get manager() {return this.department.manager;}
```



```
manager = aPerson.department.manager;
```

Problema:

Una clase tiene métodos que simplemente delegan a otros objetos. Si una clase sólo realiza una acción, delegando trabajo a otra clase, ¿por qué existe?

Solución:

Elimine estos métodos y obligue al “cliente” a llamar directamente a los métodos finales.

Actividad

- Acepte la tarea <https://classroom.github.com/a/ohilk9u4>
- Realice una refactorización del código
- Entrega vía commit en github hasta viernes 01/05

